

AllianceMine Release Process

End-to-end process for building and releasing a new AllianceMine version.

Overview

[Build on AllianceMineDev] -> [Tomcat container on Multitenant] -> [Cutover]

The build runs on **AllianceMineDev** (172.31.60.197), produces a versioned database on RDS, populates versioned Solr cores on the Multitenant, and the WAR is deployed to a parallel Tomcat container on the **Multitenant** (172.31.59.87). The release is then a port swap or proxy update.

Prerequisites

- SSH access to both machines via ~/.ssh/AGR-ssl3.pem
- AWS CLI configured (for S3 sync and CloudWatch storage monitoring)
- Solr running on Multitenant at :8983 with cores pre-created (see Step 3)
- Sufficient RDS storage (~250 GB free for build + checkpoints)

Configuration

The .env file on AllianceMineDev (~/agr_intermine_builder/docker/alliancemine/.env) needs:

```
RDS_HOST=intermine-postgres.cmnhlso7wdi.us-east-1.rds.amazonaws.com
RDS_PORT=5432
RDS_USER=postgres
RDS_PASSWORD=<secret>

ALLIANCE_RELEASE=9.0.0
FMS_RELEASE_TYPE=current
BUILD_TYPE=test

IMAGE_TAG=9.0.0

# SGD Database (external, read-only) - get current password from SGD
SGD_DB_HOST=www-rds-primary.yeastgenome.org
SGD_DB_NAME=sgd
SGD_DB_USER=webb
SGD_DB_PASSWORD=<get from SGD>
SGD_DB_PORT=5432

# Solr cores on multitenant - use versioned core names
SOLR_INDEX_URL=http://172.31.59.87:8983/solr/alliancemine-search-9.0.0
SOLR_AUTOCOMPLETE_URL=http://172.31.59.87:8983/solr/alliancemine-autocomplete-9.0.0
```

Never commit credentials. All secrets stay in .env only.

Step 1: Pre-build Cleanup

Before starting a new build, free up RDS storage:

```
# SSH to dev machine
ssh -i ~/.ssh/AGR-ssl3.pem ec2-user@172.31.60.197
```

```
# List existing RC databases
export PGPASSWORD='<password>'
psql -h intermine-postgres.cmnhlso7wdi.us-east-1.rds.amazonaws.com -U postgres -d
↳ postgres \
  -c "SELECT datname, pg_size_pretty(pg_database_size(datname)) FROM pg_database WHERE
  ↳ datname LIKE 'alliancemine_%' ORDER BY pg_database_size(datname) DESC"

# Drop old RC databases (keep production version)
psql -h ... -d postgres -c "DROP DATABASE alliancemine_X_X_X_rcN"
```

Check RDS free storage:

```
aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name FreeStorageSpace \
  --dimensions Name=DBInstanceIdentifier,Value=intermine-postgres \
  --start-time $(date -u -d '5 minutes ago' +%Y-%m-%dT%H:%M:%S) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%S) \
  --period 300 --statistics Average \
  --query 'Datapoints[0].Average' --output text | awk '{printf "%.0f GB free\n",
  ↳ $1/1024/1024/1024}'
```

Need at least **250 GB free** (build is ~57 GB, plus 4-6 checkpoint copies).

Step 2: Build the Image and Compile

```
ssh -i ~/.ssh/AGR-ssl3.pem ec2-user@172.31.60.197
cd ~/agr_intermine_builder/docker/alliancemine
git pull origin refactor/alliancemine-docker

# Build image (fast, ~5 min)
docker compose build

# Start a container to trigger the deferred Gradle compilation (~10-15 min)
docker compose run -d --rm alliancemine-builder

# Wait for compilation to finish, then commit as compiled image
CNAME=$(docker ps --format '{{.Names}}' | grep builder)
# Wait until /root/.needs_compile is gone
while docker exec $CNAME ls /root/.needs_compile 2>/dev/null; do sleep 30; done
docker commit $CNAME alliancemine-builder:9.0.0-compiled
docker stop $CNAME
```

The compiled image saves ~15 minutes per subsequent run.

Step 3: Run the Full Build

Edit `docker-compose.yml` to use the compiled image:

```
image: alliancemine-builder:9.0.0-compiled
```

Start the build in tmux:

```
tmux new-session -d -s build 'docker compose run --rm alliancemine-builder build 2>&1 |  
↪ tee /tmp/alliancemine-9.0.0-build.log'
```

Monitor:

```
tail -f /tmp/alliancemine-9.0.0-build.log
```

Expected duration: **5-8 hours** total - preflight (Solr cores): seconds (idempotent — skips if cores exist) - buildDB: ~30 sec - extract_data: ~5 min (warm cache) / ~25 min (cold; includes API fetchers) - project_build: 4-7 hours (xlaevis-fasta and fbbt are the bottlenecks) - postprocess: ~1 hour - war: ~5 min

The preflight creates alliancemine-search-{release}[-rc{N}] and the matching autocomplete core on the multitenant host via SSH. Failure is non-fatal — postprocess will surface a clearer “core not found” later, and you can recreate cores manually with create_solr_cores.py and resume with --start-from postprocess.

The build creates checkpoint databases at each dump="true" stage in project.xml. These are full DB copies via CREATE DATABASE ... WITH TEMPLATE. They use disk space (~30-60 GB each) but enable resume.

Step 4: Resume from a Checkpoint (if needed)

If the build fails midway:

```
# Check what checkpoint exists  
psql -h ... -d postgres -c "SELECT datname FROM pg_database WHERE datname LIKE  
↪ 'alliancemine_X_X_X_rcN%'"  
  
# Resume from last checkpoint  
docker compose run --rm -e RC_NUMBER=N alliancemine-builder build --start-from  
↪ project_build --resume
```

--resume uses project_build -l which loads the last checkpoint database and continues. Do **NOT** add -b (it would drop all checkpoints).

Step 5: Manage Checkpoints During Build

Checkpoints accumulate fast. Drop older ones once newer ones exist:

```
psql -h ... -d postgres -c 'DROP DATABASE "alliancemine_X_X_X_rcN:checkpoint-name"'
```

Quote the name with double quotes (the : requires quoting).

Step 6: Start the New Tomcat Container

On the **Multitenant** machine:

```
ssh -i ~/.ssh/AGR-ssl3.pem ec2-user@172.31.59.87  
  
# Start a new Tomcat container on a free port (e.g. 8082)  
# --add-host maps the old Solr hostname to the Docker host where Solr runs  
docker run -d --name alliancemine-9.0.0 \  
  --add-host agr.stage.alliancemine.solr.server:host-gateway \  
  -p 8082:8080 intermine-tomcat:latest
```

Step 7: Deploy the WAR via cargoRedeployRemote

On AllianceMineDev, point the build container at the new Tomcat and deploy:

```
# Start the compiled image
docker compose run -d --rm -e RC_NUMBER=N alliancemine-builder
CNAME=$(docker ps --format '{{.Names}}' | grep builder)

# Update properties to point at the new Tomcat
docker exec $CNAME sh -c '
    sed -i "s|webapp.deploy.url=.*|webapp.deploy.url=http://172.31.59.87:8082|"
    ↪ /root/alliancemine/alliancemine.properties
    sed -i "s|webapp.hostname=.*|webapp.hostname=172.31.59.87|"
    ↪ /root/alliancemine/alliancemine.properties
    sed -i "s|webapp.port=.*|webapp.port=8082|" /root/alliancemine/alliancemine.properties
'

# Deploy
docker exec $CNAME sh -c 'cd /root/alliancemine && ./gradlew cargoRedeployRemote
    ↪ --stacktrace'
```

Step 8: Fix Runtime Properties on the Deployed Webapp

The WAR has build-time properties baked in. Several need fixing post-deploy:

```
ssh -i ~/.ssh/AGR-ssl3.pem ec2-user@172.31.59.87

# Fix profile DB to point to production (not test)
docker exec alliancemine-9.0.0 sed -i \
    's|databaseName=alliancemine_userprofile_test|databaseName=alliancemine_userprofile|' \
    /usr/local/tomcat/webapps/alliancemine/WEB-INF/classes/intermine.properties

# Fix superuser to match production profile DB
docker exec alliancemine-9.0.0 sed -i \
    's|superuser.account=admin@alliancemine.org|superuser.account=superuser@mail_account|;
    ↪ s|superuser.password=admin|superuser.initialPassword=secret|' \
    /usr/local/tomcat/webapps/alliancemine/WEB-INF/classes/intermine.properties

# Fix webapp.baseurl in BOTH properties files
docker exec alliancemine-9.0.0 sed -i \
    's|webapp.baseurl=http://localhost:8090|webapp.baseurl=http://localhost:8080|' \
    /usr/local/tomcat/webapps/alliancemine/WEB-INF/classes/intermine.properties

docker exec alliancemine-9.0.0 sed -i \
    's|webapp.baseurl=http://localhost:8090|webapp.baseurl=http://localhost:8080|' \
    /usr/local/tomcat/webapps/alliancemine/WEB-INF/web.properties

# Stop and start the webapp (reload doesn't reread properties properly)
docker exec alliancemine-9.0.0 sh -c \
    'curl -s -m 30 -u manager:manager
    ↪ http://localhost:8080/manager/text/stop?path=/alliancemine'
docker exec alliancemine-9.0.0 sh -c \
    'curl -s -m 180 -u manager:manager
    ↪ http://localhost:8080/manager/text/start?path=/alliancemine'
```

Step 9: Verify

```
# Service version endpoint
docker exec alliancemine-9.0.0 sh -c 'curl -s
↳ http://localhost:8080/alliancemine/service/version'
# Expected: 35 (or current InterMine version)

# Solr cores have data
docker exec alliancemine sh -c 'curl -s
↳ "http://172.31.59.87:8983/solr/alliancemine-search-9.0.0/select?q=*:*&rows=0"'
# Expected: numFound > 12 million
```

Bag upgrade thread will run on first start (upgrading user gene lists from old DB IDs). This blocks queries for ~10-30 min depending on list count. Be patient.

Step 10: Cutover

To switch from old to new on production:

1. Verify the new Tomcat is healthy and queries work
2. Update Caddy proxy config to route from `https://alliancemine.alliancegenome.org` -> port 8082
3. Or stop the old container and rename ports

The bluegenes and caddy containers don't need changes — they connect to alliancemine by container name.

Rollback

If the new release has issues: 1. Revert Caddy proxy back to old port 2. The old container is still running on its original port 3. The old database is still on RDS (we never delete the previous version's DB during cutover)

What Goes Where (Quick Reference)

Setting	Location	Notes
RDS credentials	.env on dev	Never commit
SGD DB credentials	.env on dev	Never commit
Solr URLs	.env on dev -> docker-compose -> properties template	Versioned per release
Database name	Auto-derived from ALLIANCE_RELEASE and RC_NUMBER	alliancemine_X_X_X_rcN
Profile DB	alliancemine_userprofile (production) or alliancemine_userprofile_test	Production for releases
Superuser account	superuser@mail_account (in production profile DB)	Don't use the template default